МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки і комп’ютерних технологій

Звіт

про виконання лабораторної роботи №9

з курсу «Вибрані розділи науки про дані»

«Збереження знань у графовій базі даних»

Виконав:
студент групи ФЕІм-13с
Шевчук Максим Вікторович

Перевірив:
Парубочий В. О.

Львів 2026

Тема та мета роботи

Тема — збереження знань у графовій базі даних на прикладі аналізу та візуалізації електроспоживання. Мета — доповнити наявну графову базу знань запитами Cypher, дослідити її структуру та зіставити відповіді й час виконання трьох однакових питань у SPARQL і Cypher.

Завдання

1. Перевірити модель графа, обмеження та індекси.
2. Підтвердити ідемпотентність імпорту й відповідність джерелу.
3. Виконати різні типи запитів Cypher та перевірити PROFILE.
4. Дослідити ієрархію, найкоротші шляхи та ступені вузлів.
5. Порівняти SPARQL і Cypher та обґрунтувати вибір сховища.

Використання результатів лабораторної роботи №8

Модель, імпортер та повний граф я створив у своїй лабораторній роботі №8. Надалі «ЛР08» означає мій звіт «Збереження знань у графовій базі даних для візуалізації енергетичних даних», 2026, 20 с. Замість повторного опису цих частин посилаюся на відповідні розділи та сторінки мого звіту.

Нові обчислення виконано локально: Neo4j Community 5.26.32, Python 3.11.6, драйвер neo4j 5.28.2, RDFLib 7.1.4. Чотири нові результати додатково підтверджено скриншотами Neo4j Sandbox. Часові вимірювання нижче належать локальному запуску ЛР09, а не попередньому Sandbox-запуску ЛР08.

1 Модель графа та обмеження

Правила відображення OWL у граф наведено в ЛР08, розд. 2, с. 3; структуру класів та індивідів — у розд. 4–7, с. 5–8. Модель використано повторно. Її зв'язок TYPE_OF відповідає INSTANCE_OF із прикладу поточного завдання.

Обмеження Lab8Entity.id та індекси описано в ЛР08, розд. 10, с. 11. Додатковий SHOW CONSTRAINTS підтвердив унікальність id у Sandbox (рис. 1). Назва її типу тут — NODE_PROPERTY_UNIQUENESS; локально Neo4j повертає UNIQUENESS.



Рис. 1 — Обмеження унікальності id у Neo4j Sandbox

2 Ідемпотентний імпорт

Пакетний MERGE повторно використано з ЛР08, розд. 3, с. 4, додаток А, с. 15–16. Новий локальний повтор підтвердив сталість лічильників: створено 0 нових вузлів і 0 ребер (табл. 1).

Таблиця 1 — Контроль джерела та проєкції

Показник	**Результат**
Початкові RDF-трійки	993; Turtle ізоморфний OWL
Класи / початкові індивіди	37 / 26; ID збігаються
Повний граф після двох імпортів	43 746 вузлів / 199 287 ребер

Групи міток і типів ребер збігаються з джерелом. Число RDF-трійок не дорівнює числу ребер: частина значень стає властивостями; часові дані ЛР01 розширюють граф.

3 Запити Cypher

Базові 11 сценаріїв ЛР08 наведено в її розд. 8, с. 9, та додатку В, с. 19–20. Нові запити доповнюють їх OPTIONAL MATCH, collect, обмеженням кількості рядків та аналізом шляхів. Тексти нових запитів подано в додатку А цього звіту; Q9 повторює Q8 ЛР08.

Таблиця 2 — Питання та фактичні відповіді

Код	Питання	Результат
Q1	Які індивіди мають метрику?	26 рядків; для 25 — null
Q2	Які засоби обліку має об’єкт?	count = 2; meter, submeter
Q3	Які найбільші прогнози понад поріг за добу?	2,206241 та 1,951941 kWh
Q4	Які всі предки класу прогнозу?	3 предки; відстані 1, 2, 3
Q5	Які нащадки DomainEntity?	36 нащадків
Q8	Які поняття мають найбільші ступені?	5 класів; максимум 45
Q9	Яка добова MAE прогнозів?	4 прогнози; 0,429294 kWh

Параметри: $householdId = "owl:household"; $start = "2009-09-09T00:00:00"; $end = "2009-09-10T00:00:00"; $threshold = 1,5613 kWh. Інтервал [start, end) не включає праву межу; час джерела не має UTC-зсуву.

У Q1 збережено індивідів без метрики. Q2 повертає count і впорядкований collect. Q3 використовує WHERE, ORDER BY та LIMIT 5; оскільки умовам відповідають лише два прогнози, отримано два рядки. Q9 з avg використано з ЛР08 без повторного опису алгоритму.

Таблиця 3 — Нові локальні плани PROFILE

Запит	DB Hits	Початковий доступ
Q3	13	NodeIndexSeekByRange
Q9	63	NodeIndexSeekByRange

4 Шляхи та аналіз графа

4.1 Предки та нащадки класів

У ЛР08, розд. 4, с. 5, показано прямі зв’язки ієрархії. Новий Q4 обходить SUBCLASS_OF*1.. і повертає всіх предків HighConsumptionForecast, а не лише безпосереднього батька. Отримано ForecastResult на відстані 1, AnalysisResult — 2, DomainEntity — 3.

Об’єднання повернених шляхів містить чотири вузли та три ребра, що підтверджує рис. 2. Q5 аналогічно знаходить 36 нащадків DomainEntity. Обидві відповіді звірено з незалежним обходом ієрархії з файлу relationships.json.

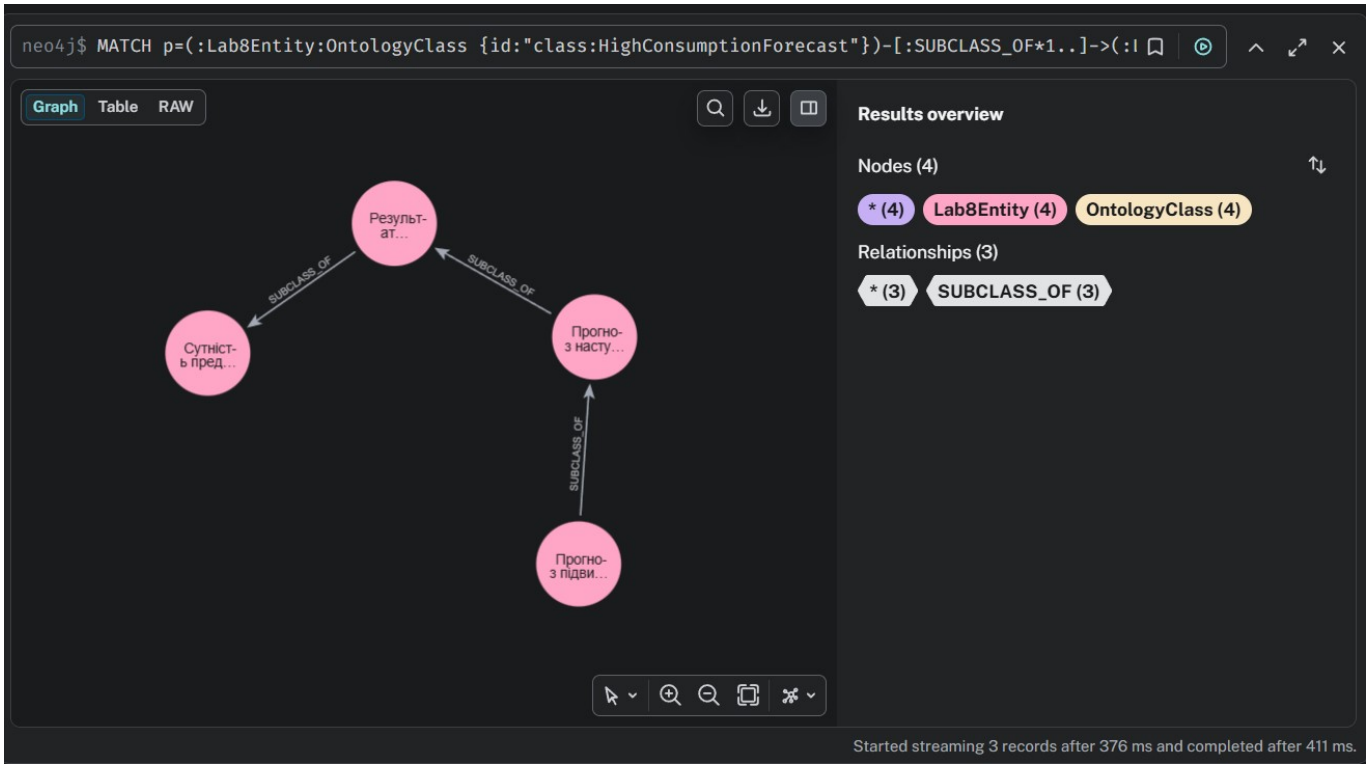


Рис. 2 — Усі предки HighConsumptionForecast у Sandbox

Відсутність верхньої межі довжини тут не створює невизначеності: збережена ієрархія містить 37 класів і 36 ребер. Запит відновлює явно задані шляхи цієї моделі; нове OWL-виведення він не запускає.

4.2 Найкоротші шляхи

Q6 шукає shortestPath між owl:analyst та owl:kWh у підграфі 26 початкових індивідів за предметними відношеннями. Напрямок ребер під час обходу не обмежується. Мінімальна довжина — 4 ребра.

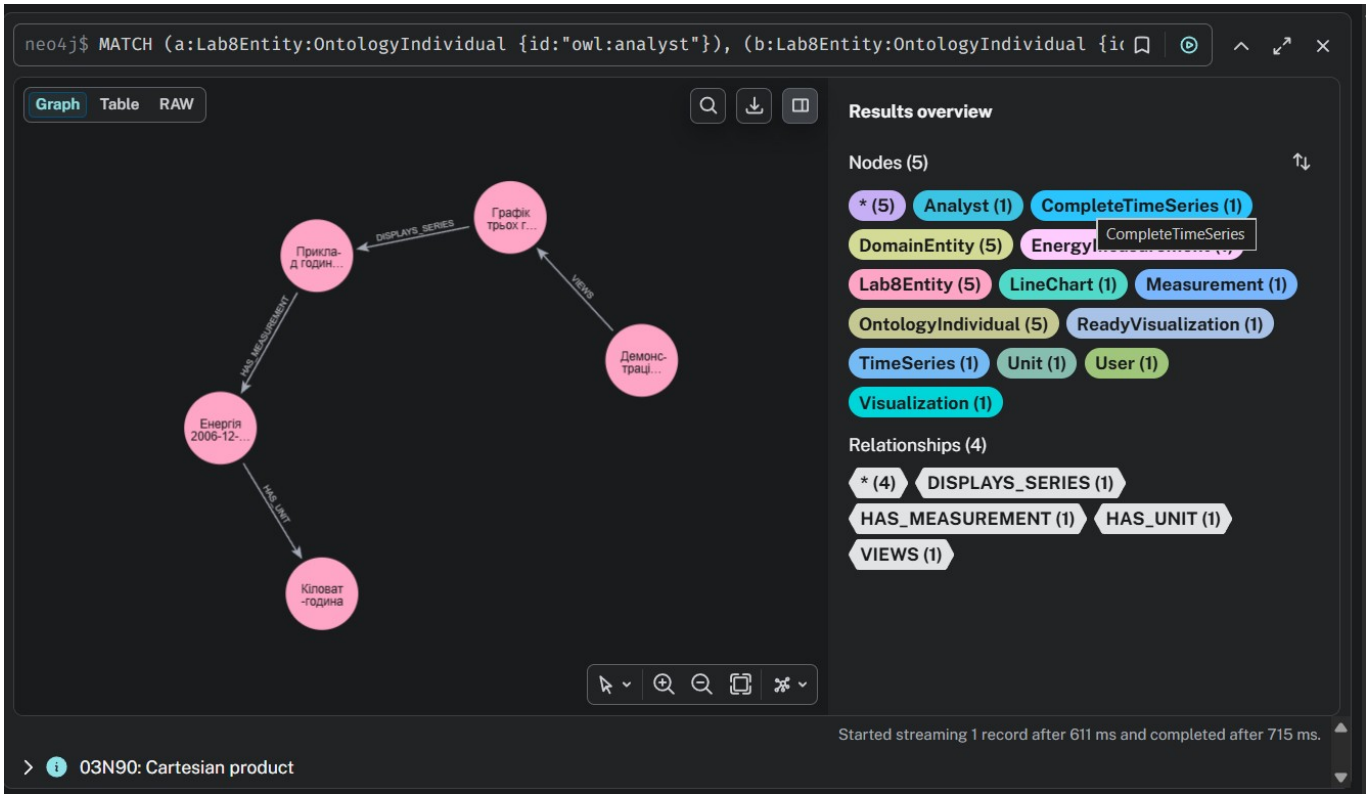


Рис. 3 — Найкоротший шлях у підграфі початкових індивідів

На рис. 3 аналітик переглядає графік, графік відображає ряд, ряд містить вимірювання, а вимірювання має одиницю kWh: VIEWS → DISPLAYS_SERIES → HAS_MEASUREMENT → HAS_UNIT. ID вимірювання в кадрі не видно, тому конкретну годину не встановлювали.

Локально отримано інший рівнозначний маршрут: analyst → lineChart → household ← targetMeasurement → kWh. Незалежний пошук у ширину також дав довжину 4.

У повному графі Q11 знаходить 2 ребра TYPE_OF: analyst → DomainEntity ← kWh. Цей шлях відображає спільний клас. Довжина 4 стосується лише визначеного предметного підграфа.

03N90 Cartesian product — інформаційна нотифікація: два кінці вибрано за унікальними id, тобто поєднується одна пара вузлів. Запит повернув один запис.

4.3 Ступені вузлів та найбільш зв’язані поняття

Ступінь визначено як число інцидентних ребер усіх типів, без обмеження напрямку. Це не кількість різних сусідів. Q8 обмежує початкову множину вузлами OntologyClass; результати на рис. 4 збігаються з незалежним підрахунком за файлом ребер.

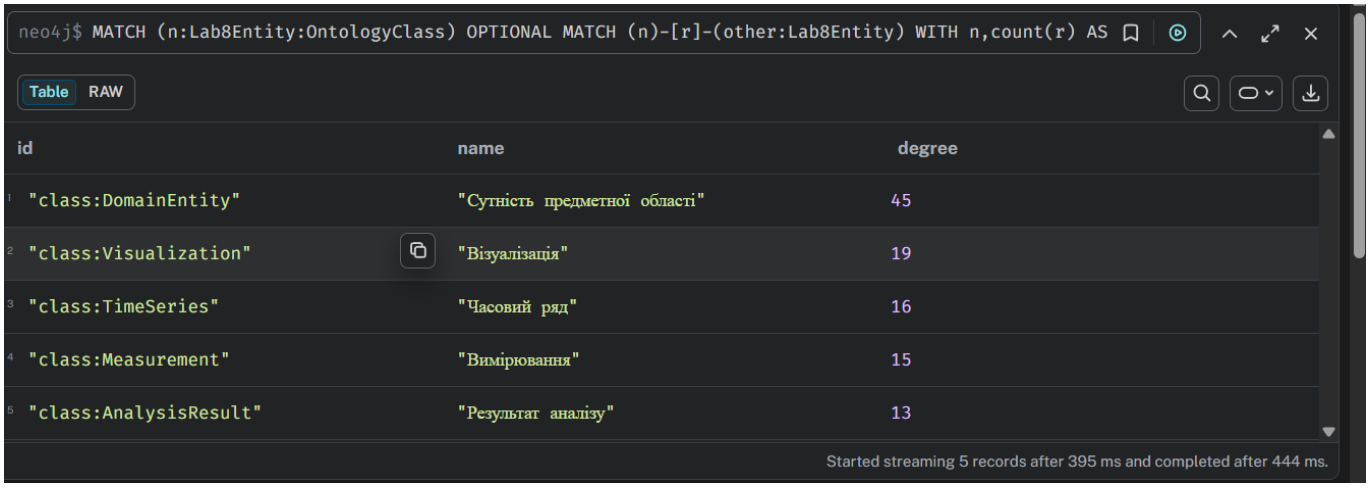


Рис. 4 — Топ-5 онтологічних понять за ступенем у Sandbox

Найбільш зв’язаний клас — DomainEntity, ступінь 45; далі Visualization — 19, TimeSeries — 16, Measurement — 15, AnalysisResult — 13. Показник включає не лише SUBCLASS_OF, а й зв’язки типізації, domain/range, що інцидентні класу.

Таблиця 4 — Топ-5 вузлів у повному графі за Q7

Вузол	**Ступінь**
data:hourlySeries	43619
owl:household	34102
owl:meter	34097
owl:energy	34094
owl:kWh	34091

На рівні всього графа переважають часовий ряд і спільні контекстні сутності вимірювань. Тому топ вузлів повної вибірки та топ онтологічних класів відповідають різним питанням.

5 Порівняння SPARQL та Cypher

5.1 Однакові питання та збіг відповідей

Питання SPARQL із ЛР05 доповнено еквівалентами Cypher у ЛР09 (додаток Б). Онтологію ЛР06 і її графову проєкцію взято з матеріалів ЛР08.

Порівнюються ті самі 26 початкових індивідів. RDF має 993 початкові трійки, а зі збереженими фактами HermiT і належністю до надкласів — 1335. Це узгоджено з матеріалізованими типами й відношеннями Neo4j; нового запуску reasoner не було.

Таблиця 5 — Парні питання та узгоджені відповіді

Пара	**Питання**	**Результат обох мов**
C1	Які засоби обліку належать домогосподарству?	meter, submeter; 2 рядки
C2	Які модель, прогноз і метрика пов’язані між собою?	forecast → regressor; 1 рядок
C3	Які подання мають тип ForecastVisualization?	forecastChart; 1 рядок

У C2 прогноз дорівнює 1,9519405504171758 kWh; MAE — 0,3359660136113589 kWh для всіх 9530 тестових прогнозів ЛР01 за 09.09.2009–26.11.2010. Ця MAE має інше охоплення, ніж добові чотири прогнози Q9.

Відповіді збігаються за IRI, значеннями й кількістю рядків; допуск для чисел — 10^{-9}. Без збережених висновків C2 і C3 у RDF повертають 0 рядків. SPARQL сам не запускає OWL reasoner.

5.2 Методика вимірювання

Для кожної пари й рушія виконано 3 прогріви та 10 повторів із чергуванням порядку. Вимірювали виконання й повне отримання відповіді в Python. Читання графа, prepareQuery, перші виконання та PROFILE виключено.

5.3 Час виконання та вибір подання знань

Таблиця 6 — Медіани локального часу за 10 повторами

Пара	SPARQL, ms	Cypher, ms	Рядків
C1	0,1854	1,5707	2
C2	0,2815	1,5368	1
C3	0,1958	1,6754	1

SPARQL виконується в RDFLib у процесі Python, Cypher — у Neo4j через Bolt. RDF містить аксіоми та збережені висновки; Neo4j — також повні часові дані. Через різні накладні витрати й фізичні обсяги ці часи характеризують лише наведені реалізації.

Сирі 60 повторів збережено в language_samples.csv, відповіді й статистики — у language_comparison.json запуску local_20261016T190455229366Z. В усіх повторах відповіді залишалися еквівалентними.

Читабельність та виразність

У C1 обидва записи короткі: hasMeter відповідає HAS_METER. У C2 SPARQL описує дані трійками, Cypher — шаблонами ребер і властивостями. Перший зручний для IRI та RDF-термів, другий наочно показує переходи. SPARQL 1.1 також підтримує property paths.

Семантика та рекомендація

Обмеження проєкції OWL описано в ЛР08, розд. 2, с. 3. У C3 тип ForecastVisualization доступний завдяки збереженому висновку; для нових даних його слід актуалізувати окремо. Автоматичного виконання всіх OWL-аксіом у цій проєкції немає.

Для контексту вимірювань, походження прогнозів та обходів доцільна Neo4j. OWL/RDF варто залишити джерелом семантичної моделі й обміну; перевірку узгодженості та нове виведення виконувати reasoner окремо. Це відповідає структурі застосунку.

Відтворення результатів

README містить версії, порядок запуску й поділ середовищ. Джерела ЛР08 збережено без змін із SHA-256. Онтологія представлена в ontology_asserted.ttl та ontology_semantic.ttl. Випадкова генерація не використовується; seed не потрібен.

Запуск run_lab9.py без --import-data читає наявний граф. Для нової бази --import-data виконує імпорт через MERGE двічі та перевіряє сталість лічильників. Облікові дані вводяться інтерактивно й у файли результатів не записуються. Основні алгоритми порівняння наведено в додатку В; імпортер повторно використано з ЛР08, додаток А, с. 15–16.

Висновок

Доповнено граф ЛР08 новими запитами Cypher. Знайдено 3 предки HighConsumptionForecast, 36 нащадків DomainEntity та найкоротші шляхи з урахуванням обраного підграфа; топ-5 класів підтверджено в Sandbox. Три пари SPARQL/Cypher дали однакові відповіді. Модель, імпорт та індекси використано з ЛР08 за наведеними посиланнями; нові вимірювання характеризують локальні реалізації.

Додаток А Нові запити Cypher

queries/*.cypher

Нумерація Q1–Q11 відповідає номерам файлів у queries/. Повторно використаний Q9 наведено в ЛР08, додаток В, с. 20, тому його лістинг тут не дублюється.

Q1 Які онтологічні екземпляри мають метрику оцінювання, а які — ні?

```
MATCH (n:Lab8Entity:OntologyIndividual)
OPTIONAL MATCH (n)-[:EVALUATED_BY]->(metric:Lab8Entity)
RETURN n.id AS id, n.name AS name, metric.id AS metric_id,
       metric.metricValue AS metric_value
ORDER BY id;
```

Q2 Які засоби обліку належать домогосподарству?

```
MATCH (h:Lab8Entity:OntologyIndividual {id:$householdId})
      -[:HAS_METER]->(m:Lab8Entity:OntologyIndividual)
WITH h,m ORDER BY m.id
RETURN h.id AS household, count(m) AS meter_count,
       collect(m.id) AS meters, collect(m.name) AS meter_names;
```

Додаток А Нові запити Cypher продовження

Q3 Які п’ять найбільших прогнозів понад поріг за обрану добу?

```
MATCH (f:Lab8Forecast)
WHERE f.targetTime >= localdatetime($start)
  AND f.targetTime < localdatetime($end)
  AND f.forecastKwh > $threshold
RETURN f.id AS id, toString(f.targetTime) AS timestamp,
  f.forecastKwh AS forecast_kwh
ORDER BY forecast_kwh DESC, id
LIMIT 5;
```

Q4 Які всі предки класу прогнозу підвищеного споживання?

```
MATCH p=(c:Lab8Entity:OntologyClass {id:$classId})
        -[:SUBCLASS_OF*1..]->(a:Lab8Entity:OntologyClass)
RETURN DISTINCT a.id AS ancestor, a.name AS name, length(p) AS hops
ORDER BY hops, ancestor;
```

Q5 Які всі нащадки кореневого класу DomainEntity?

```
MATCH p=(c:Lab8Entity:OntologyClass)
        -[:SUBCLASS_OF*1..]->(root:Lab8Entity:OntologyClass
          {id:$rootId})
RETURN DISTINCT c.id AS descendant, c.name AS name, length(p) AS
  hops
ORDER BY hops, descendant;
```

Додаток А Найкоротші шляхи

Q6 Який найкоротший шлях від аналітика до kWh у підграфі 26 початкових індивідів?

```
MATCH (a:Lab8Entity:OntologyIndividual {id:$pathStart}),
      (b:Lab8Entity:OntologyIndividual {id:$pathEnd})
MATCH p=shortestPath((a)-[:ANALYZES_SERIES|
  BASED_ON_MODEL|
  BELONGS_TO_DATASET|
  CONFIGURES|
  CONTAINS_SERIES|
  COVERS_INTERVAL|
  CREATES_VISUALIZATION|
  DERIVED_FROM_SERIES|
  DISPLAYS_SERIES|
  EVALUATED_BY|
  GENERATED_BY|
  HAS_MEASUREMENT|
  HAS_METER|
  HAS_UNIT|
  IMPLEMENTED_BY|
  INSTALLED_AT|
  MAKES_DECISION|
  MEASUREMENT_OF|
  OBSERVES_INDICATOR|
  PRESENTED_IN|
  PRESENTS_RESULT|
  PROCESSES|
  PRODUCES_RESULT|
  RECORDED_BY|
  RECORDS|
  SERIES_INDICATOR|
  SERIES_OF|
  SUPPORTS_DECISION|
  USES_MODULE|
  VIEWS|
  VISUALIZATION_OF*1..8]-(b))
WHERE all(n IN nodes(p) WHERE n:OntologyIndividual)
RETURN p, length(p) AS hops,
       [n IN nodes(p) | n.id] AS node_ids,
       [r IN relationships(p) | type(r)] AS relationship_types;
```

Q11 Який найкоротший шлях від аналітика до kWh, якщо дозволити також вузли класів?

```
MATCH (a:Lab8Entity {id:$pathStart}), (b:Lab8Entity {id:$pathEnd})
MATCH p=shortestPath((a)-[*1..8]-(b))
RETURN p, length(p) AS hops, [n IN nodes(p) | n.id] AS node_ids,
       [r IN relationships(p) | type(r)] AS relationship_types;
```

Додаток А Ступені вузлів та підграф

Q7 Які п’ять вузлів мають найбільшу кількість інцидентних ребер у повному графі?

```
MATCH (n:Lab8Entity)
OPTIONAL MATCH (n)-[r]-(other:Lab8Entity)
WITH n,count(r) AS degree
RETURN n.id AS id, n.name AS name, labels(n) AS labels, degree
ORDER BY degree DESC,id
LIMIT 5;
```

Q8 Які п’ять онтологічних понять найбільш зв’язані у повному графі?

```
MATCH (n:Lab8Entity:OntologyClass)
OPTIONAL MATCH (n)-[r]-(other:Lab8Entity)
WITH n,count(r) AS degree
RETURN n.id AS id, n.name AS name, degree
ORDER BY degree DESC,id
LIMIT 5;
```

Q10 Як виглядає підграф усіх предків обраного класу?

```
MATCH p=(:Lab8Entity:OntologyClass
  {id:$classId})-[:SUBCLASS_OF*1..]->(:Lab8Entity:OntologyClass)
RETURN p;
```

Додаток Б Парні запити SPARQL та Cypher

C1 Які засоби обліку належать домогосподарству?

comparison/C1_household_meters.rq

```
PREFIX energy: <https://example.org/lnu/energy-visualization#>
SELECT ?meter WHERE { energy:household energy:hasMeter ?meter }
  ORDER BY ?meter
```

comparison/C1_household_meters.cypher

```
MATCH (:Lab8Entity:OntologyIndividual
  {id:$householdId})-[:HAS_METER]->(m:Lab8Entity:OntologyIndividual)
RETURN m.iri AS meter ORDER BY meter;
```

C3 Які подання мають тип ForecastVisualization з урахуванням збережених висновків?

comparison/C3_inferred_visualization.rq

```
PREFIX energy: <https://example.org/lnu/energy-visualization#>
SELECT ?view WHERE { ?view a energy:ForecastVisualization } ORDER BY
  ?view
```

comparison/C3_inferred_visualization.cypher

```
MATCH (v:Lab8Entity:OntologyIndividual:ForecastVisualization)
RETURN v.iri AS view ORDER BY view;
```

Додаток Б Походження прогнозу та метрика

C2 Які модель, прогноз і метрика оцінювання пов’язані між собою?

comparison/C2_forecast_provenance.rq

```
PREFIX energy: <https://example.org/lnu/energy-visualization#>
SELECT ?forecast ?model ?value ?mae ?scope WHERE {
  ?forecast a energy:ForecastResult; energy:generatedBy ?model;
    energy:forecastKwh ?value; energy:evaluatedBy ?metric .
  ?metric energy:metricValue ?mae; energy:evaluationScope ?scope
} ORDER BY ?forecast ?model
```

comparison/C2_forecast_provenance.cypher

```
MATCH
  (f:Lab8Entity:OntologyIndividual:ForecastResult)-[:GENERATED_BY]-
>(m:Lab8Entity:OntologyIndividual),
      (f)-[:EVALUATED_BY]->(metric:Lab8Entity:OntologyIndividual)
RETURN f.iri AS forecast, m.iri AS model, f.forecastKwh AS value,
       metric.metricValue AS mae, metric.evaluationScope AS scope
ORDER BY forecast,model;
```

IRI в обох відповідях порівнюються повністю. Префікс energy позначає https://example.org/lnu/energy-visualization#. Числові RDF-літерали value і mae перед зіставленням перетворюються на числа; їхні початкові RDF-терми збережені окремо.

Додаток В Виконання та вимірювання SPARQL

run_lab9.py — основні фрагменти нового порівняння

```
def sparql_run(graph, prepared, numeric_columns):
    started = time.perf_counter()
    result = graph.query(prepared)
    names = [str(v) for v in result.vars]
    rows, terms = [], []
    for record in result:
        values = dict(zip(names, record))
        rows.append({k: (None if v is None else float(v) if k in
numeric_columns else str(v))
                     for k, v in values.items()})
        terms.append({k: None if v is None else v.n3() for k, v in
values.items()})
    return rows, terms, (time.perf_counter() - started) * 1000
```

Чергування порядку та контроль незмінності відповіді

```
for iteration in range(repeats):
    order = ['sparql', 'cypher'] if iteration % 2 == 0 else ['cypher',
'sparql']
    for engine in order:
        if engine == 'sparql':
            rows, _, wall_ms = sparql_run(semantic, prepared,
pair['numeric_columns'])
        else:
            rows, _, wall_ms = run(session, pair['cypher'], params)
        if not equivalent(sparql_rows, rows):
            raise RuntimeError('Answers changed during benchmarking: ' +
identifier)
        samples.append({'pair': identifier, 'engine': engine, 'iteration':
iteration + 1,
                        'wall_ms': wall_ms, 'returned_rows': len(rows)})
```